

Relatório: T3 IA

Brunna Moura, Carlos Cruz, Rafael Queiroz, Victor Bitencourt

Definição do Problema

O problema consiste na criação de uma aplicação RAG que consiga retornar output estruturado, com mecanismos de validação, mostrando que uma solução de IA pode ser utilizada em sistemas maiores, integrada a fluxos existentes.

Base Documental

A base documental consiste no relatório da CPMI de 8 de janeiro de 2023, quando houve tentativa de golpe de Estado no Brasil. A base documental foi escolhida estrategicamente, tratando de um assunto possivelmente conhecido por modelos de IA, no entanto, esperando-se no presente trabalho que a mesma traga fontes para sustentar seus resultados.

Pipeline de Ingestão

A pipeline de ingestão é feita ao executar o módulo `src/run_ingestion.py`. É feito o carregamento do text do documento PDF, segmentação dos *chunks*, o *embedding* do seu conteúdo, e a inserção das passagens com seus *embeddings* associados dentro do banco PostgreSQL.

Arquitetura da Solução

A pipeline de ingestão consiste em um módulo `loader.py` que extrai o conteúdo do texto do PDF utilizando a biblioteca `PyMuPDF`. É sabido que ela não é adequada para todos os tipos de documentos, já que a mesma depende fortemente da existência das tags do PDF internamente ao documento para fornecer output de qualidade. No entanto, para o documento selecionado, o `PyMuPDF` se saiu muito bem, retornando uma quantidade de conteúdo e *chunks* coerente com o tamanho do documento.

Sobre os *chunks*, é feita uma segmentação com tamanho do *chunk* de 2000 caracteres, com a biblioteca `langchain-text-splitters`. O *overlap* é de 200 caracteres. Foram escolhidos esses valores após inspeção manual de como ficaria o tamanho da passagem para mandar para a IA posteriormente. Foi considerado que, para o presente documento, os valores usuais de 500-1000 caracteres com um *overlap* minúsculo não serviriam. O ideal seria um *chunking* semântico, mas não foi possível detectar seções utilizando a biblioteca `PyMuPDF` no passo anterior de carregamento de arquivo.

Sobre os *embeddings*, foi selecionado o modelo `pt_core_news_lg` da biblioteca `spaCy`, não correspondente ao mesmo provedor de LLM utilizado - é um modelo local, não via API, como o modelo de LLM do Gemini usado nesse trabalho.

Sabe-se que a mesma utiliza embeddings `word2vec`, utilizando uma arquitetura de redes neurais convolucionais, o que resulta em embeddings de menor qualidade se comparado com modelos que utilizam a arquitetura Transformers. No entanto, esse modelo foi selecionado principalmente devido à maior facilidade de integração, já que um modelo local de embedding não apresentaria desafios no que diz respeito à implementação de políticas de *backoff* no caso de um possível esbarro no limite de solicitações, o que é comum em modelos de IA via API. Em particular, a Gemini API, utilizada pela equipe para o modelo generativo de LLM, estabelece limites muito restritos de solicitações diárias, o que dificultaria muito os testes pela equipe no caso de alcance de um limite de API dessa natureza.

Sobre o modelo de LLM, foi selecionado o `gemini-3.1-flash-lite`. É o melhor modelo econômico fornecido pela plataforma, tendo um excelente desempenho para tarefas mais simples e diretas como perguntas-e-respostas via RAG. Conecta-se ao modelo via biblioteca `langchain-google-genai`.

O armazenamento persistente das passagens e *chunks* é realizado pelo módulo `domain/cpmidoc/service.py`, que realiza a conexão com o banco PostgreSQL e fornece os métodos de acesso, customizados para retornar nossos modelos a serem usados pela aplicação.

A tabela SQL possui o seguinte esquema.

```
CREATE TABLE chunk (  
    id INTEGER PRIMARY KEY GENERATED ALWAYS AS IDENTITY,  
    snippet TEXT NOT NULL,  
    embedding halfvec(300) NOT NULL,  
    page INTEGER NOT NULL  
);
```

A dimensionalidade de valor 300 é a suportada pelo modelo de embedding escolhido. Foi escolhido o tipo `halfvec` da extensão PGVector do PostgreSQL, para otimizar o armazenamento com perda de precisão mínima, agilizando os testes, a partir de uma estratégia de quantização escalar.

Para a busca semântica, uma similaridade de cosseno comum é aplicada, utilizando os operadores usuais fornecidos pela extensão PGVector.

```
SELECT snippet, page, embedding <=> %s::halfvec AS distance  
FROM chunk  
ORDER BY distance  
LIMIT %s;
```

A busca semântica compara o embedding da pergunta do usuário com o que está no banco e retorna as passagens relevantes, de acordo com um valor de top-k correspondente. Foi estabelecido um valor de k=5 para os testes.

O formato do prompt é montado manualmente, com a pergunta do usuário no final. Foi feito dessa forma por ser uma boa prática: prompts no final com partes fixas no início se beneficiam do *prompt caching* fornecido pelas APIs. A seguir,

o template de prompt montado pela equipe, em um formato XML, que costuma ser o formato mais bem entendido pelas LLMs, dado que delimita os limites de cada seção com tags, o que é satisfatório para a forma como a LLM processa texto: de forma linear.

```
<system>
O contexto a seguir vem de busca semântica.
Responda o usuário com base no contexto retornado.
Há a possibilidade do contexto não ser relevante,
dado que vem de uma busca semântica direta.
</system>
<context>
{"\n\n".join(search_results)}
</context>
<user>
{query}
</user>
```

É mais prático fazer dessa forma do que com uma ferramenta passada para o modelo. Primeiro, porque o presente trabalho não se dispõe a fornecer uma interface a fazer um fluxo de chat, onde a busca semântica seria opcional. Em segundo lugar, modelos não suportam output estruturado junto com ferramentas, o que necessitaria de modelos separados, cada para realizar uma das tarefas, por exemplo, que seria complicação demais para o que se propõe.

A validação do objeto retornado já é feita internamente pelo framework **langchain**, utilizado no trabalho. No presente trabalho, a implementação de validação é feita no caso de erros de solicitação à API do modelo. Em termos qualitativos, o formato do objeto a ser retornado para aplicação, correspondente à resposta da IA com as fontes, que consiste no output estruturado proposto no trabalho, aceita nenhuma fonte como resultado. Essa é a forma com que a aplicação lida com respostas possivelmente não encontradas. A seguir, o formato de objeto Pydantic esperado que o LLM retorne.

```
from pydantic import BaseModel, Field

class AIAnswer(BaseModel):
    content: str = Field(description="Sua resposta")
    sources: str | None = Field(description="Todas as fontes para embasar a resposta")
```

Modelos e Componentes Utilizados

Conforme mencionado, o modelo de LLM via API **gemini-3.1-flash-lite** e o modelo de embeddings local utilizando **word2vec**, o **pt_core_news_lg**.

Bibliotecas: **langchain-google-genai** para se conectar à API do Gemini, **langchain-text-splitters** para realizar o *chunking*, **psycopg** para se conectar ao banco PostgreSQL, **pymupdf** para carregar o conteúdo do PDFem *string*, e

spaCy para carregar o modelo de embeddings local.

Protocolo Experimental

Resultados

Resumo de Desempenho (Acertos/Total)

Categoria	Acertos	Total
casos faceis	3	6
casos medios	4	7
casos ambiguos	3	6
casos onde a base de conhecimento e insuficiente	5	6
casos que testam os limites da aplicacao	5	5
Total	20	30

Análise Crítica

Conclusão

Considerações sobre uso de IA

Referências

geminiapi scalar word2vec